

Yes. Below is a **BA + SRS starter package** for the proposed **AI MIS & ERP Copilot**. I've structured it so you can use it as a real software-project document and later convert it into development tasks.

AI MIS & ERP Copilot

Business Analysis (BA) & Software Requirements Specification (SRS)

Version: 1.0

Project Type: AI-powered Enterprise MIS / ERP Assistant

Architecture: ASP.NET Core + Vue + SQL Server + LLM + RAG

Primary Users: Management, MIS, Finance, Operations, IT, Branch/Regional users

1. Business Analysis (BA)

1.1 Executive Summary

AI MIS & ERP Copilot is an AI-powered enterprise application that allows authorized users to interact with ERP/MIS data using natural language.

Instead of manually writing SQL queries, navigating multiple reports, or asking the IT/MIS team for data, users can ask questions such as:

"Show me the total loan collection for August 2026."

"Which branches have collection below recoverable by more than 10%?"

"Show me the top 10 branches based on savings collection."

"Why is Branch 0212 showing high overdue?"

The system interprets the question, identifies the relevant business entities and data sources, generates a safe SQL query, executes it against authorized datasets, and presents the result through tables, charts, and AI-generated explanations.

2. Business Problem

Current ERP/MIS environments commonly have several challenges:

2.1 Data access complexity

Users need to navigate multiple screens and reports to obtain information.

2.2 Dependency on IT/MIS

Business users frequently depend on technical personnel for customized reports.

2.3 SQL knowledge requirement

Many useful business questions require SQL knowledge.

2.4 Report generation time

Creating ad-hoc reports can take considerable time.

2.5 Data interpretation

Users may receive raw numbers without an explanation of what those numbers mean.

2.6 Multiple data sources

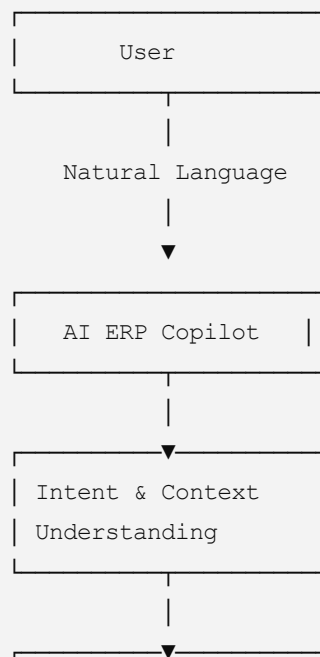
ERP systems may contain separate databases/modules such as:

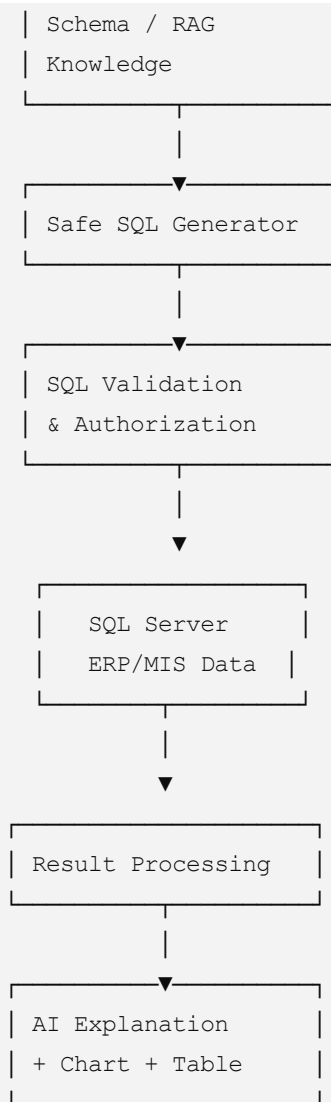
```
ERP_MIS
ERP_HRM
ERP_ACCOUNTING
ERP_FMS
ERP_INV
ERP_SECURITY
```

Finding relationships between these systems can be difficult for nontechnical users.

3. Proposed Business Solution

The proposed system introduces an AI layer between users and ERP/MIS data.





4. Business Objectives

Primary objectives

1. Reduce dependency on IT/MIS teams for routine data queries.
 2. Allow business users to retrieve ERP information using natural language.
 3. Reduce the time required to create ad-hoc reports.
 4. Improve management decision-support capabilities.
 5. Provide AI-generated explanations of MIS data.
 6. Improve accessibility of complex ERP information.
 7. Maintain strict authorization and data security.
 8. Create an auditable record of AI-generated queries and results.
-

5. Business Benefits

Before

```
User
↓
Request MIS report
↓
MIS/IT
↓
Understand requirement
↓
Write SQL
↓
Test SQL
↓
Generate report
↓
Send to user
```

After

```
User
↓
Ask AI
↓
AI understands requirement
↓
Generate safe query
↓
Execute
↓
Table / Chart / Summary
```

Potential benefits:

- Faster reporting
- Reduced manual effort
- Improved self-service analytics
- Better management visibility
- Reduced repetitive SQL/report requests
- Consistent business definitions
- Better utilization of existing ERP data

6. Stakeholder Analysis

Stakeholder	Responsibility
Management	View business insights

Stakeholder	Responsibility
MIS Team	Validate reports/business rules
Finance	Financial analysis
Operations	Operational analysis
Branch Users	Branch-level information
IT Team	System administration
Database Administrator	Database/security management
Business Analyst	Requirements/business rules
Developer	Application development
System Administrator	Infrastructure

7. User Roles

7.1 System Administrator

Can:

- Manage users
- Manage roles
- Manage AI configuration
- Manage data sources
- Manage permissions
- View audit logs

7.2 MIS Administrator

Can:

- Configure business terminology
- Configure report definitions
- Manage database metadata
- Validate AI-generated queries
- Review AI accuracy

7.3 Management User

Can:

- Ask business questions
- View dashboards
- Generate reports
- Export reports

7.4 Branch User

Can only access authorized branch data.

Example:

```
Branch User
  ↓
BranchCode = 0212
  ↓
AI Query
  ↓
Only Branch 0212 data
```

7.5 Regional/Area User

Can access data according to assigned Area/Region.

8. Major Use Cases

UC-01: Ask Business Question

Actor: Authorized User

Example

User:

"What was the total collection during August 2026?"

System:

1. Receives question.
2. Detects intent = Collection Report.
3. Identifies date range.
4. Identifies relevant database/table.
5. Generates SQL.
6. Validates SQL.

7. Applies user access restrictions.
 8. Executes query.
 9. Displays result.
 10. Generates explanation.
-

9. UC-02: Branch Performance Analysis

User:

"Show branches where collection is less than 90% of recoverable."

AI generates a query conceptually equivalent to:

SQL

```
SELECT
    BranchCode,
    SUM(RecoverableAmount) AS RecoverableAmount,
    SUM(CollectionAmount) AS CollectionAmount,
    SUM(RecoverableAmount) -
        SUM(CollectionAmount) AS GapAmount
FROM ...
GROUP BY BranchCode
HAVING
    SUM(CollectionAmount)
    <
    SUM(RecoverableAmount) * 0.90;
```

The system then produces:

Branch	Recoverable	Collection	Gap	Collection %
0212	12,500,000	9,800,000	2,700,000	78.40%
0215	8,200,000	7,000,000	1,200,000	85.37%

10. UC-03: Generate Chart

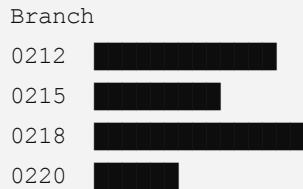
User:

"Show August collection by branch."

AI identifies:

Dimension = Branch
Measure = Collection
Period = August 2026
Chart = Bar Chart

Result:



11. UC-04: Explain Report

User:

"Explain this report."

AI should explain:

- What the report measures
- Date range
- Number of records
- Major changes
- Highest/lowest values
- Important anomalies
- Business interpretation

The AI must distinguish **calculated facts** from interpretations.

12. UC-05: Follow-up Question

The conversation should maintain context.

User

Show August loan collection.

AI

Total collection was ٦X.

User

What about July?

The system should understand that "What about July?" refers to the same collection metric.

13. UC-06: Export Report

Supported formats:

- Excel
- CSV
- PDF

Example:

```
AI Query
  ↓
Result
  ↓
Export
├─ Excel
├─ CSV
└─ PDF
```

14. UC-07: AI Report Builder

User:

"Create a monthly loan performance report."

The system creates:

```
Monthly Loan Performance

1. Total Proposal
2. Total Disbursement
3. Total Recoverable
4. Total Collection
5. Outstanding
6. Overdue
7. Collection Rate
8. Branch-wise performance
```

15. Key Business Rules

BR-001 — Authorization

AI must never bypass the application's existing authorization mechanism.

BR-002 — Branch restriction

A branch user must only see authorized branch data.

For example:

```
User Branch = 0212
```

```
Generated Query
```

```
↓
```

```
Security Filter
```

```
↓
```

```
BranchCode = 0212
```

BR-003 — Read-only AI database access

The initial version should allow:

```
SELECT
```

and prohibit:

```
INSERT
```

```
UPDATE
```

```
DELETE
```

```
DROP
```

```
ALTER
```

```
TRUNCATE
```

```
EXEC
```

unless explicitly approved through a separate controlled mechanism.

BR-004 — SQL validation

Every AI-generated SQL query must pass validation before execution.

BR-005 — Business terminology

The AI should understand organization-specific terminology.

Example:

```
Recoverable
Collection
Outstanding
Overdue
Disbursement
Savings
Center
Member
Loan Product
Branch
EIN
```

16. Functional Requirements — SRS

FR-001 Authentication

The system shall provide secure user authentication.

Possible implementation:

```
JWT
OAuth 2.0
ASP.NET Core Identity
```

FR-002 User Management

Admin shall be able to:

- Create user
 - Update user
 - Activate/deactivate user
 - Assign role
 - Assign branch
 - Assign region
 - Assign permissions
-

FR-003 Natural Language Query

The system shall allow users to enter natural-language questions.

Example:

```
"How much was collected in August 2026?"
```

FR-004 AI Intent Detection

The system shall determine:

```
Intent
Entity
Metric
Date
Filters
Grouping
Sorting
```

Example:

```
Question:
"Show branch-wise August loan collection."

Intent:
CollectionReport

Entity:
Branch

Metric:
CollectionAmount

Period:
2026-08-01 → 2026-08-31

Group:
Branch
```

17. FR-005 Schema Understanding

The AI shall have access to metadata rather than unrestricted database knowledge.

Example:

Table:
LoanRecoverable

Columns:
BranchCode
AccountNo
RecoverableAmount
RecoverableDate
IsActive

18. FR-006 SQL Generation

The AI shall generate SQL based on:

- User question
- Database schema
- Business definitions
- User authorization
- Conversation context

19. FR-007 SQL Validation

Before execution:

```
AI SQL
  ↓
Parser
  ↓
Read-only validation
  ↓
Table whitelist
  ↓
Column validation
  ↓
Authorization validation
  ↓
SQL execution
```

20. FR-008 Query Execution

The system shall execute validated queries against configured read-only database connections.

21. FR-009 Result Visualization

Depending on result type, the system shall automatically select:

- Table
- Bar chart
- Line chart
- Pie/donut chart
- KPI card

22. FR-010 AI Summary

The system shall generate a concise summary from query results.

Example:

August 2026 loan collection was ₦125.4 million across 185 branches. The average collection rate was 91.3%.

23. FR-011 Conversation History

The system shall store:

Conversation ID
User ID
Question
Generated SQL
Execution time
Result metadata
AI response
Timestamp

24. FR-012 Feedback

Users shall be able to select:

-  Correct
-  Incorrect

Optional:

What was wrong?

This can later be used to improve the AI system.

25. FR-013 Query Audit

Every AI-generated SQL query must be logged.

Example:

User:

25052

Question:

Show August collection for branch 0212

Generated SQL:

SELECT ...

Execution:

Success

Execution Time:

1.82 sec

Timestamp:

2026-09-16 18:30

26. FR-014 Report Saving

Users shall be able to save frequently used queries.

Example:

My Reports

└─ Monthly Collection

└─ Branch Performance

└─ Loan Outstanding

└─ Savings Collection

└─ Overdue Analysis

27. FR-015 Scheduled Reports

Future version:

```
Every day 9:00 AM
    ↓
Run report
    ↓
Generate Excel/PDF
    ↓
Email management
```

28. FR-016 RAG Knowledge Base

The system should support organizational documents.

Examples:

```
PDF
DOCX
Excel
Policy
Manual
SOP
Business Rules
Product Definition
MIS Documentation
```

The AI can answer:

"What is the business definition of recoverable?"

or:

"According to the loan policy, what is the applicable rule?"

29. RAG Architecture

```
Documents
    ↓
Text Extraction
    ↓
Chunking
    ↓
Embedding
```

```
↓
Vector Database
↓
Similarity Search
↓
Relevant Context
↓
LLM
↓
Answer
```

30. SRS — Non-Functional Requirements

NFR-001 Performance

Normal queries should return within an acceptable response time, subject to database complexity and AI processing.

Target:

```
Simple query:
< 5 seconds
```

```
Complex query:
< 15 seconds
```

NFR-002 Security

The system must implement:

- HTTPS
- JWT/OAuth
- RBAC
- Database least privilege
- SQL validation
- Audit logging
- Secrets management

31. NFR-003 Reliability

If AI service fails:

AI Service unavailable

↓

Friendly error

↓

No database mutation

↓

Audit failure

32. NFR-004 Scalability

Architecture should support:

100 users

↓

500 users

↓

1000+ users

without major architectural changes.

33. NFR-005 Maintainability

Backend should follow:

Clean Architecture

SOLID

CQRS

Repository Pattern

Dependency Injection

Domain Services

34. Proposed Architecture

I recommend:

AI-MIS-Copilot

|

├─ AI.MIS.Api

|

├─ AI.MIS.Application

|

└─ AI.MIS.Domain

```
|
├─ AI.MIS.Infrastructure
|
├─ AI.MIS.AI
|
├─ AI.MIS.Persistence
|
└─ AI.MIS.Web
```

API

ASP.NET Core Web API

Frontend

Vue 3
TypeScript

Database

SQL Server

AI layer

LLM
Embeddings
RAG
Tool Calling

35. Suggested Backend Flow

POST /api/copilot/query

Request:

```
JSON

{
  "question": "Show August 2026 loan collection by branch"
}
```

Processing:

```
Controller
  ↓
Copilot Application Service
  ↓
Intent Analyzer
  ↓
Schema Retrieval
  ↓
Business Rule Retrieval
  ↓
Prompt Builder
  ↓
LLM
  ↓
SQL Validator
  ↓
Authorization Filter
  ↓
Database
  ↓
Result Analyzer
  ↓
Chart Generator
  ↓
AI Response
```

36. API Design

Authentication

http

POST /api/auth/login

Ask Copilot

http

POST /api/copilot/query

Conversation

http

GET /api/copilot/conversations

GET /api/copilot/conversations/{id}

Feedback

http

POST /api/copilot/feedback

Reports

http

GET /api/reports

POST /api/reports/save

GET /api/reports/{id}

Export

http

POST /api/reports/{id}/export

Knowledge

http

POST /api/knowledge/upload

GET /api/knowledge

DELETE /api/knowledge/{id}

37. Database Design

Core tables:

Users

Roles

UserRoles

UserBranchAccess

UserRegionAccess

CopilotConversation

CopilotMessage

CopilotQuery

CopilotQueryExecution

CopilotFeedback

```

AISchemaMetadata
AIBusinessTerm
AIBusinessRule

KnowledgeDocument
KnowledgeChunk
KnowledgeEmbedding

SavedReport
ScheduledReport

AuditLog
```

38. Example AI Schema Metadata

Instead of sending the entire SQL Server database to the LLM, maintain controlled metadata.

JSON

```
{
  "table": "LoanRecoverable",
  "description": "Stores loan recoverable amounts",
  "columns": [
    {
      "name": "BranchCode",
      "description": "Unique branch code"
    },
    {
      "name": "AccountNo",
      "description": "Loan account number"
    },
    {
      "name": "RecoverableAmount",
      "description": "Amount recoverable"
    },
    {
      "name": "RecoverableDate",
      "description": "Recoverable date"
    }
  ]
}
```

This is much safer than giving the AI unrestricted database access.

39. AI Prompt Strategy

The AI should receive controlled information:

SYSTEM:

You are an ERP MIS assistant.

RULES:

1. Generate SELECT queries only.
2. Never modify data.
3. Use only approved tables.
4. Follow user authorization.
5. Follow business definitions.
6. Never invent columns.
7. If information is insufficient, ask for clarification.
8. Return structured SQL.

40. Structured AI Output

Do not ask the AI to return arbitrary text.

Use structured output:

JSON

```
{
  "intent": "LoanCollectionReport",
  "sql": "SELECT ...",
  "chartType": "bar",
  "groupBy": "BranchCode",
  "summaryRequired": true
}
```

Then your .NET application controls execution.

41. Important Security Architecture

This is one of the most important parts of the project.

Never do this:

User

↓

LLM

↓

SQL

↓

Instead:

```
User
  ↓
LLM
  ↓
SQL
  ↓
SQL Parser
  ↓
Read-only validation
  ↓
Table whitelist
  ↓
Authorization
  ↓
Row-level security
  ↓
Query timeout
  ↓
Read-only DB
  ↓
Result
```

42. Example Real-World Scenario

User

"Give me August 2026 recoverable and collection for branch 0212."

AI determines

```
Branch = 0212
Month = August 2026
Metrics =
    Recoverable
    Collection
```

SQL layer

The application generates and validates the required query.

Result

Branch: 0212

Recoverable: ₺12,500,000

Collection: ₺9,800,000

Gap: ₺2,700,000

Collection Rate:

78.40%

AI explanation

For August 2026, Branch 0212 recorded ₺12.5 million in recoverable amount and ₺9.8 million in collection, resulting in a ₺2.7 million gap.

43. MVP Scope

Don't build everything initially.

Phase 1 — MVP

Implement:

- ✓ Login
- ✓ User authorization
- ✓ Chat interface
- ✓ Natural-language questions
- ✓ Schema metadata
- ✓ AI SQL generation
- ✓ SQL validation
- ✓ Read-only SQL execution
- ✓ Table results
- ✓ Basic charts
- ✓ AI summary
- ✓ Query history
- ✓ Audit log

Phase 2

- ✓ RAG
- ✓ PDF/DOCX knowledge base
- ✓ Business rules
- ✓ Excel export
- ✓ PDF export
- ✓ Saved reports
- ✓ Feedback

Phase 3

- ✓ Scheduled reports
- ✓ Email
- ✓ Advanced dashboards
- ✓ Anomaly detection
- ✓ Predictive analytics
- ✓ Multi-database querying

44. Development Roadmap

Sprint 1

Foundation

- Solution architecture
- Authentication
- User/role management
- Database
- Base API

Sprint 2

AI Integration

- LLM integration
- Prompt management
- Structured AI response
- Schema metadata

Sprint 3

Text-to-SQL

- SQL generation
- SQL parser
- Query validation
- Read-only execution

Sprint 4

Visualization

- Tables
- KPI cards

- Charts
- AI summary

Sprint 5

Security

- Branch filtering
- Role filtering
- Audit logging
- Query timeout
- Rate limiting

Sprint 6

RAG

- Document upload
- Embeddings
- Vector search
- Business-rule retrieval

Sprint 7

Reporting

- Excel
- PDF
- Saved reports
- Scheduled reports

45. Example Project Folder

```
AI-MIS-Copilot/  
|  
├─ src/  
| |  
| └─ AI.MIS.Api/  
|   | └─ Controllers/  
|   | └─ Middleware/  
|   └─ Program.cs  
|  
| └─ AI.MIS.Application/  
|   | └─ Copilot/  
|   | └─ Reports/  
|   └─ Users/
```

```

|   |   └─ Common/
|   |
|   └─ AI.MIS.Domain/
|       └─ Entities/
|       └─ Enums/
|       └─ Interfaces/
|
|   └─ AI.MIS.Infrastructure/
|       └─ AI/
|       └─ Database/
|       └─ Security/
|       └─ RAG/
|
|   └─ AI.MIS.Web/
|       └─ components/
|       └─ views/
|       └─ services/
|
└─ database/
    └─ Tables/
    └─ Procedures/
    └─ Views/
    └─ Seed/
|
└─ docs/
    └─ BA.md
    └─ SRS.md
    └─ Architecture.md
    └─ API.md
|
└─ README.md

```

46. Portfolio Description

You can describe the project on your CV/GitHub as:

AI MIS & ERP Copilot — An AI-powered enterprise analytics platform that enables authorized users to query ERP/MIS data using natural language. The system uses LLM-based intent understanding, controlled text-to-SQL generation, schema-aware RAG, SQL validation, role/branch-level authorization, automated visualization, and AI-generated business summaries. Built with ASP.NET Core, C#, SQL Server, Vue/TypeScript, Clean Architecture, CQRS, and modern AI technologies.

Recommended next step

I would build this as a **real .NET 8 project**, not just documentation. The logical next deliverables are:

1. **Complete SRS document**
2. **Complete BA document with 15–20 use cases**

3. **ERD/database SQL scripts**
4. **High-level + low-level architecture**
5. **ASP.NET Core .NET 8 solution structure**
6. **AI prompt architecture**
7. **Text-to-SQL implementation**
8. **SQL security/validation layer**
9. **Vue 3 AI chat UI**
10. **First working MVP with sample ERP/MIS database**

For your background, I would specifically make the first MVP work with **Loan Recoverable + Loan Collection + Branch + Member + Savings** data, because that gives the project a realistic ERP/MIS use case rather than a generic ChatGPT clone.